

RL in Quantum Applications

From control pulses to feedback and error correction

FYS4411 — Reinforcement Learning and Agentic AI

University of Oslo, Department of Physics

Quantum applications

Why reinforcement learning in quantum technology?

Quantum technology contains many sequential decision problems:

- ▶ choose a control pulse over time;
- ▶ prepare a target quantum state;
- ▶ design a high-fidelity gate;
- ▶ adapt to measurement outcomes;
- ▶ correct errors before coherence is lost;
- ▶ optimize sensing protocols under noise.

The common structure is

$$S_t \longrightarrow A_t \longrightarrow (R_{t+1}, S_{t+1}).$$

In quantum applications, the “environment” is usually the controlled quantum system, simulator, or experimental device.

What is different from standard RL examples?

In classical textbook examples, the agent often observes the state directly.

In quantum systems this is usually not true:

- ▶ the wave function is not directly observable;
- ▶ measurements are probabilistic;
- ▶ projective measurements disturb the state;
- ▶ observations may be only expectation values or bit strings;
- ▶ rewards are often available only at the end of an episode.

Therefore many quantum RL problems are naturally closer to

partially observable, stochastic, finite-horizon control.

Quantum RL definitions

RL object	Quantum meaning	Example
State S_t	physical state	ψ_t, ρ_t
Observation O_t	measured information	$\langle \sigma_z \rangle$, syndrome
Action A_t	control operation	pulse, gate, measurement
Reward R_{t+1}	task score	fidelity, energy, survival
Episode	full protocol	one pulse sequence

The lecture goal is to translate quantum problems into this table.

Once this translation is clear, the choice of algorithm becomes a second-order question.

Three kinds of quantum RL environments

In quantum technology, the RL environment can mean different things:

Type	Data come from	Main issue
Simulated	numerical time evolution	model may be imperfect
Real	quantum hardware	noisy, slow, partially observed
Hybrid	simulation + device data	simulation-to-device transfer

In all cases, the agent only interacts through the RL loop

$$O_t \longrightarrow A_t \longrightarrow (R_{t+1}, O_{t+1}).$$

The practical question is: **what information is actually available to the agent?**

Simulated environment: model-based use

The RL agent has access to the transition model used to generate the simulated dynamics.

$$\rho_{t+1} = \mathcal{E}_{a_t}(\rho_t), \quad |\psi_{t+1}\rangle = U(a_t)|\psi_t\rangle.$$

- ▶ planning or gradient-based updates can exploit the model
- ▶ sample efficiency can improve
- ▶ transfer to hardware may fail if the model is inaccurate.
- ▶ following from the last point: realistic models often have different dynamics. Model based training is likely to introduce bias.

Simulated environment: model-free use

A simulated quantum environment uses a physical model to generate transitions.

$$H(t) = H_0 + \sum_k u_k(t) H_k, \quad \rho_t \xrightarrow{A_t} \rho_{t+1}.$$

But the agent does not need access to the Hamiltonian.

It only receives sampled interaction data:

$$(O_t, A_t, R_{t+1}, O_{t+1}).$$

- ▶ the simulator is treated as a black box;
- ▶ the agent learns from repeated episodes;
- ▶ useful when we want a policy that does not depend on analytic gradients;
- ▶ closer in spirit to later deployment on hardware.

Real quantum environments

On real hardware, the environment is the experimental device itself.

The agent chooses a control,

A_t = pulse, gate, or measurement choice,

and receives only measured information,

O_t = bit string, syndrome, or expectation estimate.

This is harder than simulation because

- ▶ the quantum state is not directly observable;
- ▶ measurement outcomes are stochastic;
- ▶ measurements can disturb the state;
- ▶ rewards are often sparse and terminal;
- ▶ calibration and noise drift over time.

Quantum RL and the Markov assumption

Quantum RL is Markovian if the state is complete:

$$S_t = \rho_t, \quad \rho_{t+1} = \mathcal{T}_{a_t}(\rho_t).$$

Then the next step depends only on (S_t, A_t) .

In experiments, the agent usually sees observations, not ρ_t :

$$O_t = \langle \sigma_z \rangle_t, \quad m_t \in \{-1, +1\}, \quad \text{syndrome.}$$

So many quantum RL tasks are better explained by a partially observable Markov decision process:

$$A_t \sim \pi(\cdot \mid O_1, \dots, O_t).$$

Quantum Reinforcement Learning

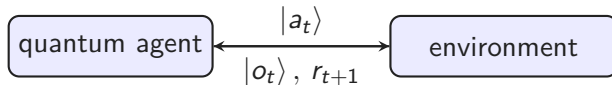
Quantum reinforcement learning studies RL settings where the agent itself is a quantum mechanical object.

In ordinary RL, percepts and actions are classical symbols:

$$o_t \in \mathcal{O}, \quad a_t \in \mathcal{A}.$$

In QRL, they may be encoded in quantum systems:

$$|o_t\rangle \in \mathcal{H}_O, \quad |a_t\rangle \in \mathcal{H}_A.$$



The question is whether quantum resources can improve learning, exploration, or decision making.

A simple classification

Let the agent type and environment type be

$$A \in \{C, Q\}, \quad E \in \{C, Q\},$$

where C means classical and Q means quantum.

Agent	Environment	Interpretation
C	C	ordinary reinforcement learning
C	Q	classical RL applied to a quantum task
Q	C	quantum-enhanced reinforcement learning
Q	Q	fully quantum reinforcement learning

(C, Q) is the usual setting in quantum applications,

(Q, C) and (Q, Q) belong more naturally to QRL, i.e. a quantum agent.

What makes the agent quantum?

In ordinary deep RL, the policy or value function is classical:

$$\pi_{\theta}(a \mid s), \quad Q_{\theta}(s, a).$$

In QRL, these objects may be represented or accelerated by quantum resources:

$$|\psi_{\theta}\rangle, \quad U_{\theta}, \quad \text{measurement outcomes.}$$

Typical quantum ingredients:

- ▶ variational quantum circuits as policies or value functions
- ▶ superposition over actions
- ▶ amplitude amplification for rewarded actions
- ▶ quantum communication between agent and environment

The goal is not just to control a quantum system, but to use quantum mechanics inside the learning agent.

Classical episodes

Consider a finite action set with losing and winning actions,

$$\mathcal{A} = \{\ell, w\}, \quad r(\ell) = 0, \quad r(w) = 1.$$

In a classical episode, the agent prepares one action state,

$$|a\rangle_A |0\rangle_R,$$

and sends it to the environment.

The environment applies a reward-marking unitary,

$$U_E |a\rangle_A |0\rangle_R = |a\rangle_A |r(a)\rangle_R.$$

The returned state is measured, producing a classical reward.

$$r \in \{0, 1\}.$$

Quantum episodes

In a quantum episode, the action register is prepared in superposition,

$$|\psi\rangle_A = \cos \xi |\ell\rangle_A + \sin \xi |w\rangle_A.$$

The reward register is prepared as

$$|-\rangle_R = \frac{|0\rangle_R - |1\rangle_R}{\sqrt{2}}.$$

The same environment unitary now phase-marks the winning action,

$$U_E |\psi\rangle_A |-\rangle_R = (\cos \xi |\ell\rangle_A - \sin \xi |w\rangle_A) |-\rangle_R.$$

A reflection step,

$$U_{\text{Ref}} = 2 |\psi\rangle \langle \psi|_A - \mathbb{I}_A,$$

amplifies the winning action before measurement.

QRL example: action and reward registers

Actions and rewards can be represented by separate registers,

$$\mathcal{H} = \mathcal{H}_A \otimes \mathcal{H}_R.$$

For a two-action, binary-reward task,

$$|a\rangle |r\rangle \in \{|0_A 0_R\rangle, |0_A 1_R\rangle, |1_A 0_R\rangle, |1_A 1_R\rangle\}.$$

The environment marks rewarded actions by

$$U_E |a\rangle |0_R\rangle = |a\rangle |r(a)\rangle.$$

Learning objective and quantum resources

The objective is still to maximize expected return:

$$J(\pi) = \mathbb{E}_{\pi} \left[\sum_{t=0}^{T-1} \gamma^t r_{t+1} \right].$$

What may change is the mechanism used to achieve this.

- ▶ **superposition of actions**

$$|\psi_A\rangle = \sum_a c_a |a\rangle$$

- ▶ **quantum environment query**

$$U_E |a\rangle |0\rangle = |a\rangle |r(a)\rangle$$

- ▶ **amplitude amplification** to increase the weight of rewarded actions

- ▶ **quantum memory or quantum policy circuits**

$$\pi_{\theta}(a \mid o) \rightsquigarrow U_{\theta}, \text{ measurement outcomes}$$

First guiding example: state preparation (from week 5)

Suppose we want to drive

$$|\psi_0\rangle \longrightarrow |\psi_{\text{target}}\rangle.$$

A simple controlled Hamiltonian may be written as

$$H(t) = H_0 + \sum_k u_k(t) H_k.$$

At each step, the agent chooses the controls

$$A_t = (u_1(t), u_2(t), \dots).$$

The terminal reward can be the fidelity

$$R_T = |\langle \psi_{\text{target}} | \psi_T \rangle|^2.$$

This is the simplest bridge from quantum optimal control to RL.

Application: Nanophotonic processor

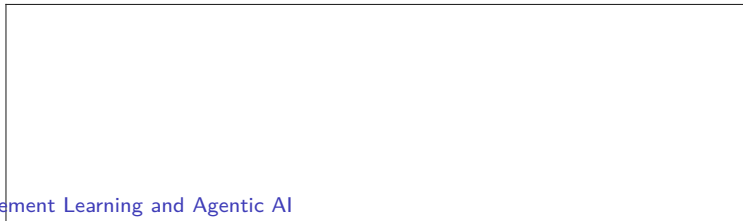
The example realizes a quantum-enhanced hybrid agent using single-photon exchange.

$$\mathcal{H} = \mathcal{H}_A \otimes \mathcal{H}_R$$

$$|0\rangle_A = |\ell\rangle_A, \quad |1\rangle_A = |w\rangle_A$$

The reward qubit marks whether the final state is winning:

$$r(a) \in \{0, 1\}.$$



Classical episode

In a classical episode, the agent prepares one action and receives a binary reward.

$$a = \begin{cases} 1, & \text{with probability } \varepsilon, \\ 0, & \text{with probability } 1 - \varepsilon. \end{cases}$$

$$\varepsilon = \sin^2 \xi = \sum_{\{a:r(a)>0\}} \pi(a | s).$$

The environment applies

$$U_E |a\rangle_A |0\rangle_R = |a\rangle_A |r(a)\rangle_R.$$

A reward is obtained when the photon is detected in the rewarding output mode.

In a quantum episode, the action register is prepared in superposition:

$$|\psi\rangle_A = \cos \xi |\ell\rangle_A + \sin \xi |w\rangle_A.$$

The reward register is prepared as $|-\rangle_R$, so that the environment acts as an oracle:

$$U_E |\psi\rangle_A |-\rangle_R = (\cos \xi |\ell\rangle_A - \sin \xi |w\rangle_A) |-\rangle_R.$$

The agent then applies

$$U_{\text{Ref}} = 2 |\psi\rangle \langle \psi|_A - \mathbb{I}_A.$$

This gives Grover-like amplitude amplification of winning trajectories.

Learning curves: why combine strategies?

figures/qrl_learning_b1.png

The quantum strategy initially increases the average reward faster.

But it peaks near an optimal episode number j^* , after which performance declines.

This motivates a hybrid strategy.

Hybrid quantum–classical strategy

figures/qrl_learning_b2.png